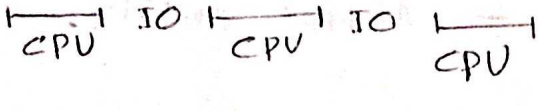
UNIT - II
Processes

1 Process Concept:

A process is a program in execution. ✓

Process execution is a cycle of CPU execution and I/O wait, beginning and ending with a CPU burst.



(Process P1)

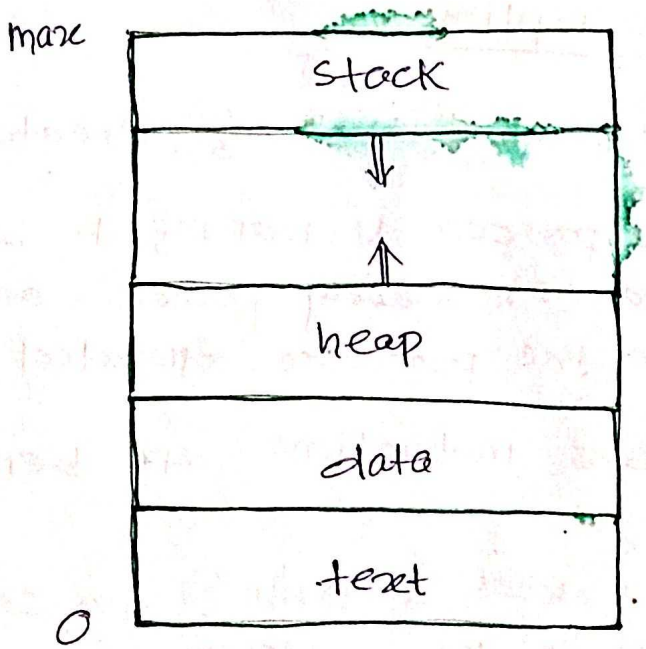
- When a process is loaded in main memory for execution, different memory area is created by OS.

(i) Text section : In text section program code is stored.

(ii) Stack : Stack contains temporary data such as function parameters, return address and local variables.

(iii) Data section : Data section contains global variables.

(iv) Heap : Heap is the memory that is dynamically allocated during process run time.



(Process in memory)

Program

- Resides in secondary memory.
- Without computing resources.
- Passive entity.

Process

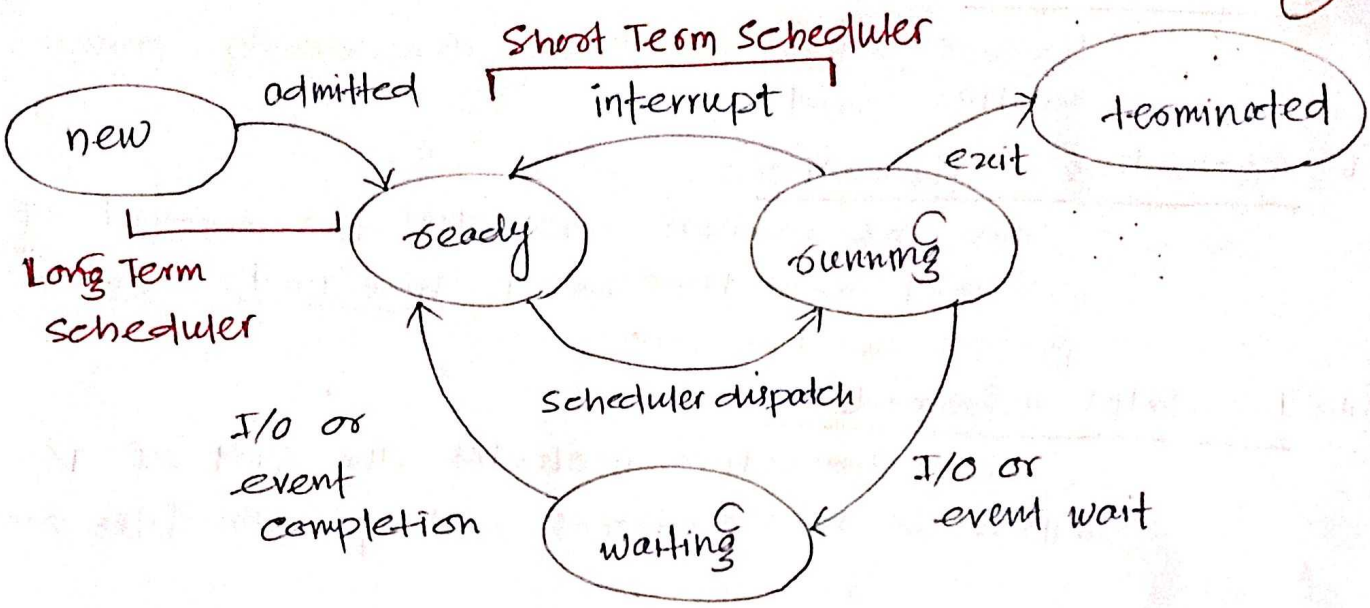
- Resides in main memory.
- Utilization of resources.
- Active entity.

- A program becomes a process when an executable file is loaded into memory.
- Although two processes may be associated with same program they are considered two different processes.

Process state

- As a process executes it changes state.
- The state of a process is defined in part by as the current activity of the process.

State	Description
New	The process is being created.
Ready	The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by OS.
Running	Process instructions are being executed
Waiting	The process is waiting for some event to occur (such as the completion of an I/O operation)
Terminated	The process has finished execution.



[Process state diagram]

Process Control Block (PCB):

Each process is represented by a PCB which is a data block containing the following information.

(i) Program counter ✓:
- indicates the address of next instruction to be executed for this process.

(ii) CPU registers: ✓
- includes stack pointer, index register and accumulator etc. number and type of register depends upon the computer architecture.

(iii) CPU scheduling information: ✓
- includes a process priority, pointers to scheduling queues and any other scheduling parameters.

(iv) Memory management information: ✓
- This information includes the value of base and limit registers, the page table or segment table.

(v) Process state :

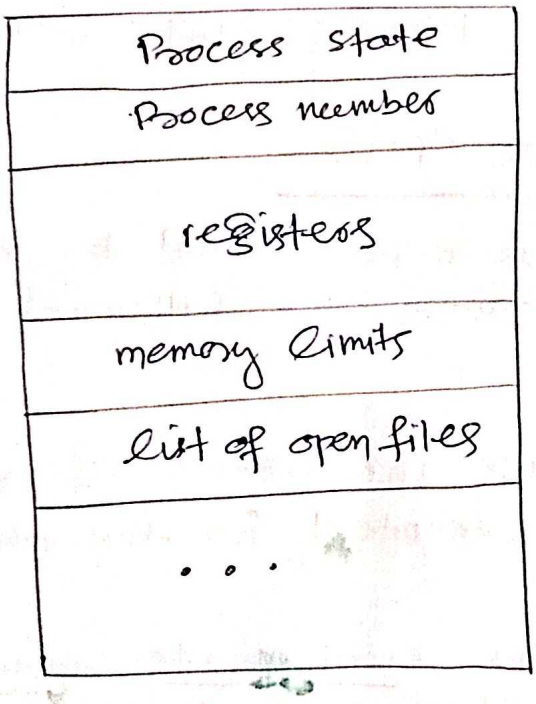
- Process state may be new, ready, running, waiting and so on.

(vi) Accounting information :

- This information includes the amount of CPU and real time used, time limits, job or process number etc.

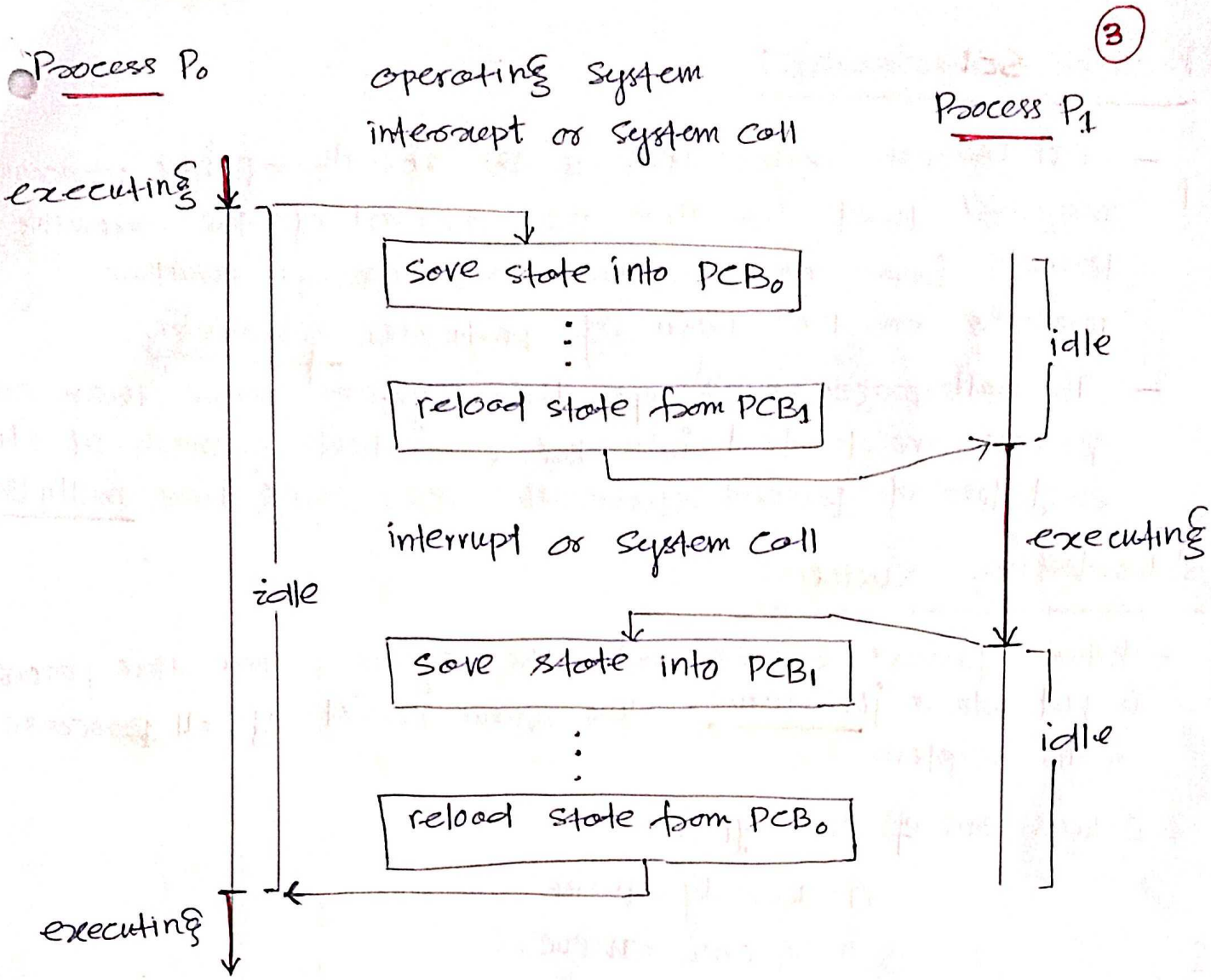
(vii) I/O status information :

- This information includes the list of I/O device allocated to the process, list of open files and so on



[Process control Block (PCB)]

- PCB is a data structure used by the OS.
- operating system groups all information that needs about particular process.



[CPU switching from process to process]

② Process Scheduling:

- The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and selection of another process on the basis of particular strategy.
- In multiprogramming operating systems more than one process are loaded into the executable memory at a time and loaded process share the CPU using time multiplexing.

Scheduling Queues:

- When a process enters into the system, then this process is put into a job queue. This queue consists of all processes in the system.

Queues are of two types:-

(i) Ready queue
(ii) Device queue.

(1) Ready queue:

- A newly arrived process is put in the ready queue.
- Once the CPU is assigned to a process then that process will execute.
- While executing the process any one of the following event can occure:

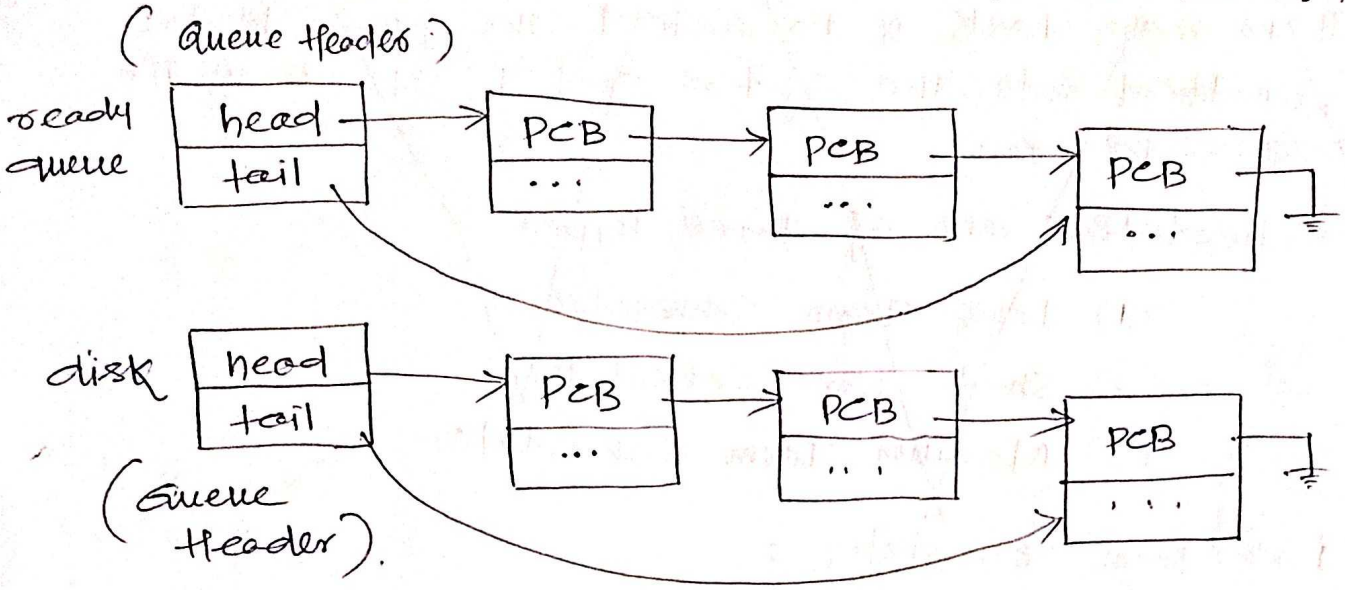
(1) The process could issue an I/O request and then it would be placed in an I/O queue.

(2) The process could create new sub process and will wait for its termination.

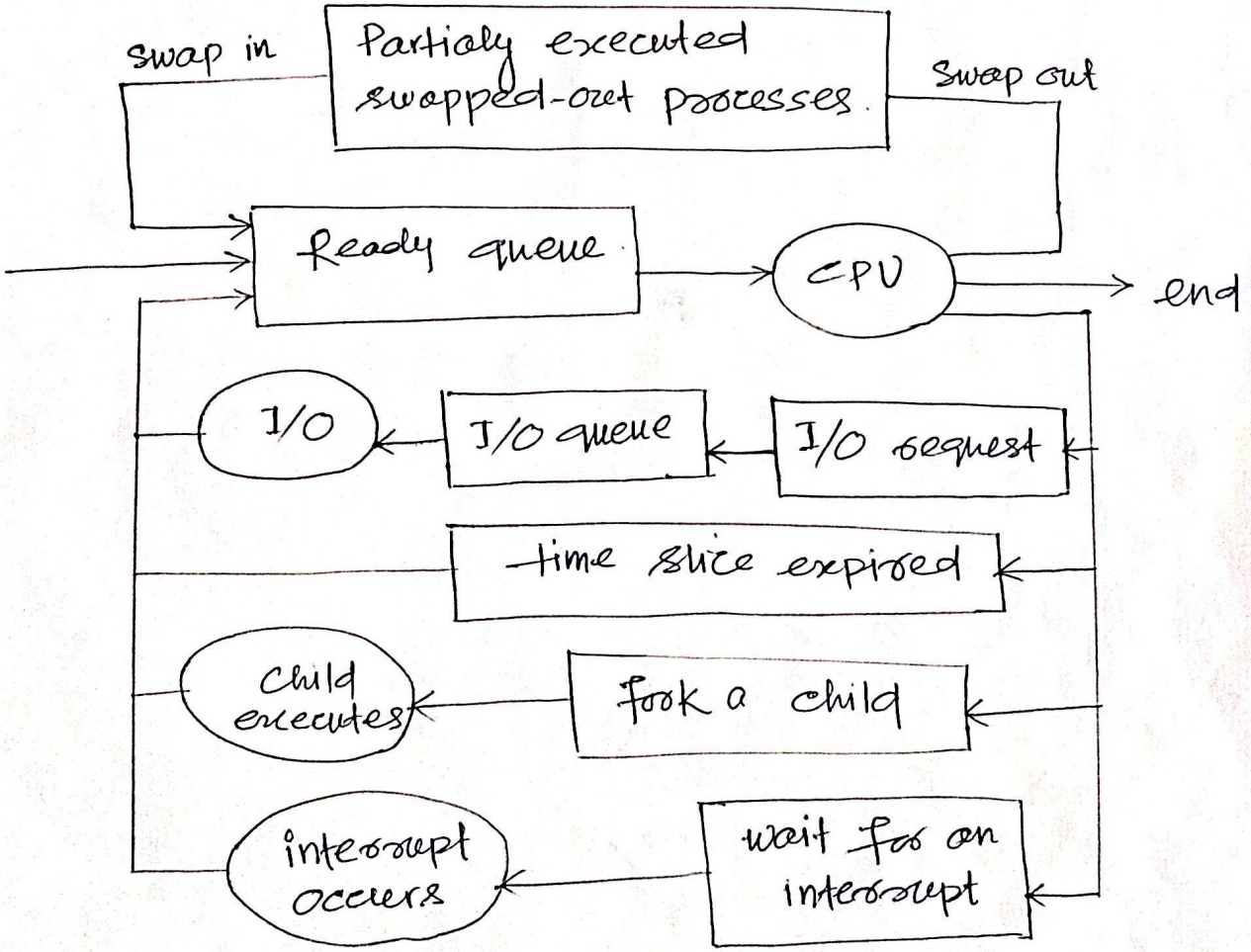
(3) The process could be removed forcibly from the CPU, as a result of interrupt and put back in the ready queue.

(iii) **Device queue :**

- Device queue contains the list of processes that are waiting for a particular I/O device
- Each device have its own device queues.



[The ready queue and I/O device queues]



[Queuing-diagram representation of process scheduling.]

Schedulers: ⑤

- Schedulers are special system software which handles process scheduling in various ways.
- The main task is to select the jobs to be submitted into the system and to decide which process to run.
- Schedulers are of three types:
  (1) Long Term Scheduler.
  (2) Short Term Scheduler.
  (3) Medium Term Scheduler.

(1) Long Term Scheduler: (Job scheduler)

- Job scheduler selects processes from the Job pool and loads them into main memory for execution.
- The primary objective of the job scheduler is to provide a balanced mix of jobs such as I/O bound and CPU bound.
- The long term scheduler controls the degree of multiprogramming (the number of processes in memory).
- If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average ~~departure~~ departure rate of processes leaving the system.

(2) Short Term Scheduler: (CPU scheduler)

- main objective is increasing system performance.
- CPU scheduler selects ~~process~~ among the processes that are ready to execute and allocates CPU to one of them
- Short term scheduler is also known as dispatcher, execute most frequently and makes the fine grained decision of which process to execute next.

(3) Medium Term Scheduler :

- To reduce the degree of multiprogramming it is required to remove processes from memory, and from active contention for the CPU.
- Later, the process can be reintroduced into memory and its execution can be continued where it left off. This process is called swapping.
- The medium term scheduler is in charge of handling the swapped out processes.

Comparision between Schedulers :

Long Term Scheduler	Short term Scheduler	Medium Term Scheduler
① It is a job scheduler.	① It is a CPU Scheduler.	① It is a swapping scheduler.
② Speed is lesser than short term scheduler.	② Speed is fastest among other two.	② Speed is between both short term and long term scheduler.
③ It controls the degree of multiprogramming.	③ No control over degree of multiprogramming	③ Reduces degree of multiprogramming.
④ It is absent ~~all~~ in time sharing system.	④ It is minimal in time sharing system.	④ It is part of time sharing system.
⑤ It selects processes from pool and loads them into memory for execution.	⑤ It selects those processes which are ready to execute.	⑤ It can reintroduce the process into memory and execution can be continued.

CPU scheduling Algorithms

Whenever the CPU becomes idle, the OS must select one process in the ready queue to be executed.

The selection process is carried out by the short term scheduler.

CPU scheduling decision may take place under the following four circumstances:-

1. When a process switches from the running state to the waiting state. (i.e I/O operation)
2. When a process switches from running state to ready state. (i.e occurance of interrupt.)
3. When a process switches from waiting state to the ready state (i.e I/O completion)
4. When a process terminates.

Scheduling algorithm can be divided into two categories.

1. **Non preemptive :**
   - A non preemptive scheduling algorithm picks a process to run and lets it run until it blocks or until it voluntarily releases the CPU.
2. **Preemptive :**
   - A preemptive scheduling algorithm picks a process and lets it run for a maximum of some fixed time.
   - If it is still running at the end of the time interval it is suspended and the scheduler picks another process to run if available.

Process Times

(1) Waiting time : Waiting time is the sum of the periods spent waiting in the ready queue.

(2) Arrival time : The time at which a job is submitted.

(3) Burst time : The time spend by process in the CPU.

(4) Turnaround time : Total time spend by the process in the system.

Completion time − Arrival time.

(5) Completion time :. The time required by the process to complete execution.

Non Preemptive Algorithms :

(1) FCFS : (First Come First serve)

The process which request first is allocated the CPU.

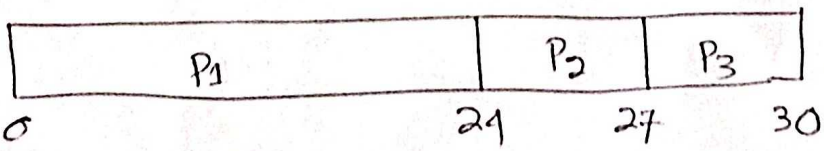
Example-(1)

Process	BT	TAT	WT
P_1	24	24	0
P_2	3	27	24
P_3	3	30	27

Average waiting time $\frac{51}{3} = 17$

Average TAT $= \frac{81}{3} = 27$

Gantt chart :



Advantage of this algorithm is simple to understand and implement.

Disadvantage: The average turnaround time and waiting time is substantial and waiting time of smaller job is more.

Example - (2)

Process	AT	BT	CT	TAT	WT
P_1	0	4	4	4	0
P_2	1	5	9	8	3
P_3	2	6	15	13	7
P_4	3	3	18	15	12
P_5	4	2	20	16	14
P_6	5	4	24	19	15

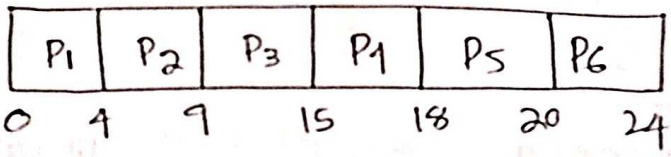
Avg TAT = $\frac{75}{6}$

12.5

Avg WT = $\frac{51}{6}$

8.5

Gantt chart:



(2) SJF: (Shortest Job First)

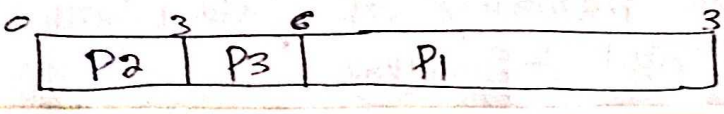
- In this case each job is associated with the length of its next CPU burst.
- When the CPU is available it is assigned to the job with the smallest next CPU burst.
- If more than one job have the same next CPU burst than FCFS is used to break the tie.

Process	BT	TAT	WT
P_1	24	30	6
P_2	3	3	0
P_3	3	6	3

Avg TAT = $\frac{39}{3}$ = (13)

Avg WT = $\frac{9}{3}$ = (3)

Gantt Chart



Advantage :

- It provides the optimal result for average TAT and WT.
- It decreases the waiting time of smaller job.

Disadvantage :

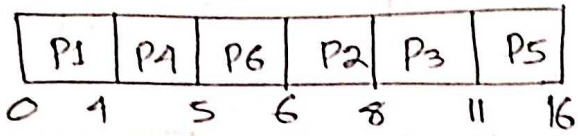
- It is based on predicted value.

Process	AT	BT	CT	TAT	WT
P1	0	4	1	1	0
P2	1	2	8	7	5
P3	2	3	11	9	6
P4	3	1	5	2	1
P5	4	5	16	12	7
P6	5	1	6	1	0

Avg TAT = $\frac{35}{6} = 5.8$

Avg WT = $\frac{19}{6} = 3.16$

Gantt chart



③ Priority :

- This algorithm associates a priority with each job.
- CPU is allocated to the job with the highest priority
- Equal priority job will served based on FCFS.

Process	Priority	BT	TAT	WT
P1	1	21	27	3
P2	0	3	03	0
P3	2	3	30	27

Avg TAT = $\frac{60}{3} = 20$

Avg WT = $\frac{30}{3} = 10$

Advantage : Priority is attached to sensitive application.

Disadvantage : It may lead to the problem of starvation.

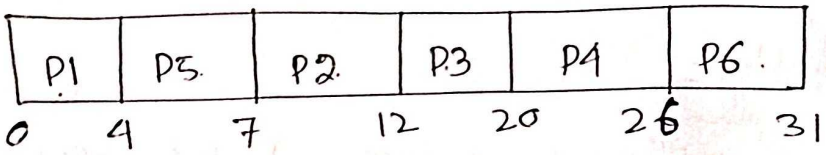
The problem of starvation can be overcome by using aging technique. It is a technique of gradually increasing the priority of job which is waiting for a long period of time.

Process	Priority	AT	BT	CT	TAT	WT
P1	4	0	4	4	4	0
P2	5	1	5	12	11	6
P3	6	2	8	20	18	10
P4	8	3	6	26	23	17
P5	2	4	3	7	3	0
P6	12	5	5	31	26	21

$$\text{Avg TAT} = \frac{85}{6} = 14.16$$

$$\text{Avg WT} = \frac{54}{6} = 9$$

Gantt chart



Preemptive Algorithms :

(1) SRTF : (Shortest Remaining time First)

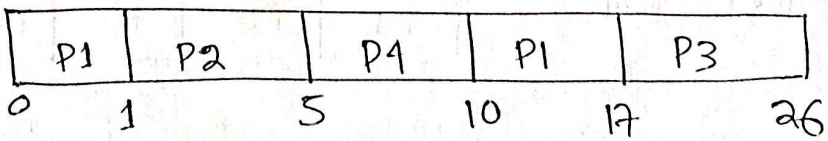
When a new job arrives at the ready queue, while the previous one is executing and the new job has a shorter next CPU burst in comparision to the left over time of the currently executing job then CPU will be preempted from the currently executing job and will be allocated to the newly arrived job.

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

$$\text{Avg TAT} = \frac{52}{4} = 13$$

$$\text{Avg WT} = \frac{26}{4} = 6.5$$

Gantt chat

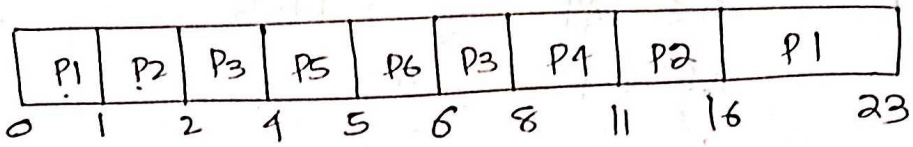


Process	AT	BT	CT	TAT	WT
P1	0	8	23	23	15
P2	1	6	16	15	9
P3	2	4	8	6	2
P4	3	3	11	8	5
P5	4	2	5	1	0
P6	5	1	6	1	0

Avg TAT $= \frac{54}{6} = 9$

Avg WT $= \frac{31}{6} = 5.16$

Gantt chart



② Priority:

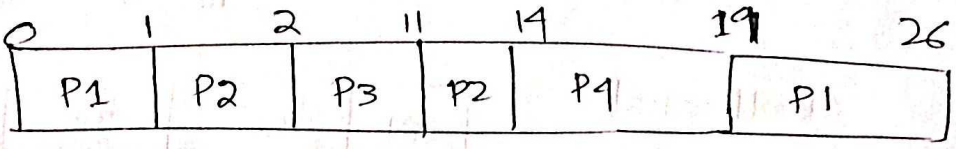
When a job arrives at the ready queue its priority is compared with the priority of the currently running job. If the priority of the new job is higher as compared to currently running job then CPU is preempted further from the current job and is allocated to the new job. Lower the value higher will be priority.

Process	AT	Priority	BT	CT	TAT	WT
P1	0	2	8	26	26	18
P2	1	1	4	14	13	9
P3	2	0	9	11	9	0
P4	3	1	5	19	16	11

Avg TAT $= \frac{64}{4} = 16$

Avg WT $= \frac{38}{4} = 9.5$

Gantt chart :



Process	Priority	AT	BT	CT	TAT	WT
P1	4	0	4	4	4	0
P2	5	1	5	12	11	6
P3	6	2	8	20	18	10
P4	8	3	6	26	23	17
P5	2	4	3	7	3	0
P6	12	5	5	31	26	21

$$\text{Avg TAT} = \frac{85}{6} = 14.17$$

$$\text{Avg WT} = \frac{54}{6} = 9$$

(3) Round Robin (RR):

This algorithm defines a Time Quantum (TQ). The ready queue Q is made a circular queue. The CPU scheduler goes arround the ready Q allocating the CPU to each process for a TQ. If CPU burst is less than TQ then the process will voluntarily release the CPU for the next job. If the CPU burst is greater than TQ then these will be interrupt. CPU will be preempted from the current process and allocated to the next job in ready queue.

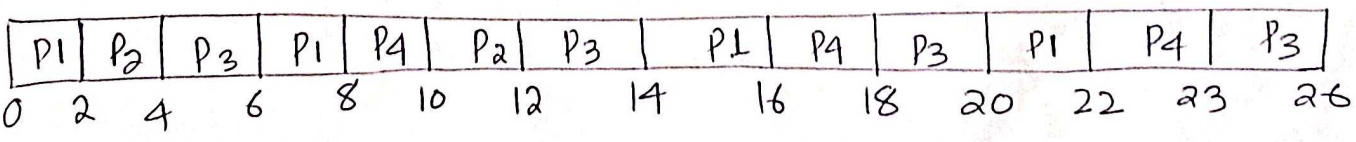
Process	AT	BT	CT	TAT	WT
P1	0	8	22	22	14
P2	1	4	12	11	7
P3	2	9	26	24	15
P4	3	5	23	20	15

TQ = 2

$$\text{Avg TAT} = \frac{77}{4} = 19.2$$

$$\text{Avg WT} = \frac{51}{4} = 12.75$$

Gantt chart



Advantage: It gives equal priority to each job.

Disadvantage: If the value of the time quantum (TQ) is large then it is equivalent to FCFS.

If the TQ is very small then there will be a very frequent context switching rather than execution.

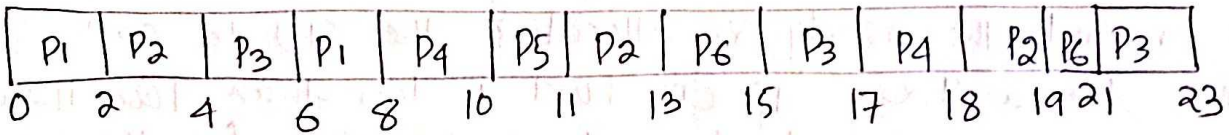
TQ = 2

Process	AT	BT	CT	TAT	WT
P1	0	4	8	8	4
P2	1	5	19	18	13
P3	2	6	23	21	15
P4	3	3	18	15	12
P5	4	1	11	7	6
P6	5	4	21	16	12

Avg TAT = $\frac{85}{6}$ = 14.16

Avg WT = $\frac{62}{6}$ = 10.33

RQ: P1 P2 P3 P1 P4 P5 P2 P6 P3 P4 P2 P6 P3



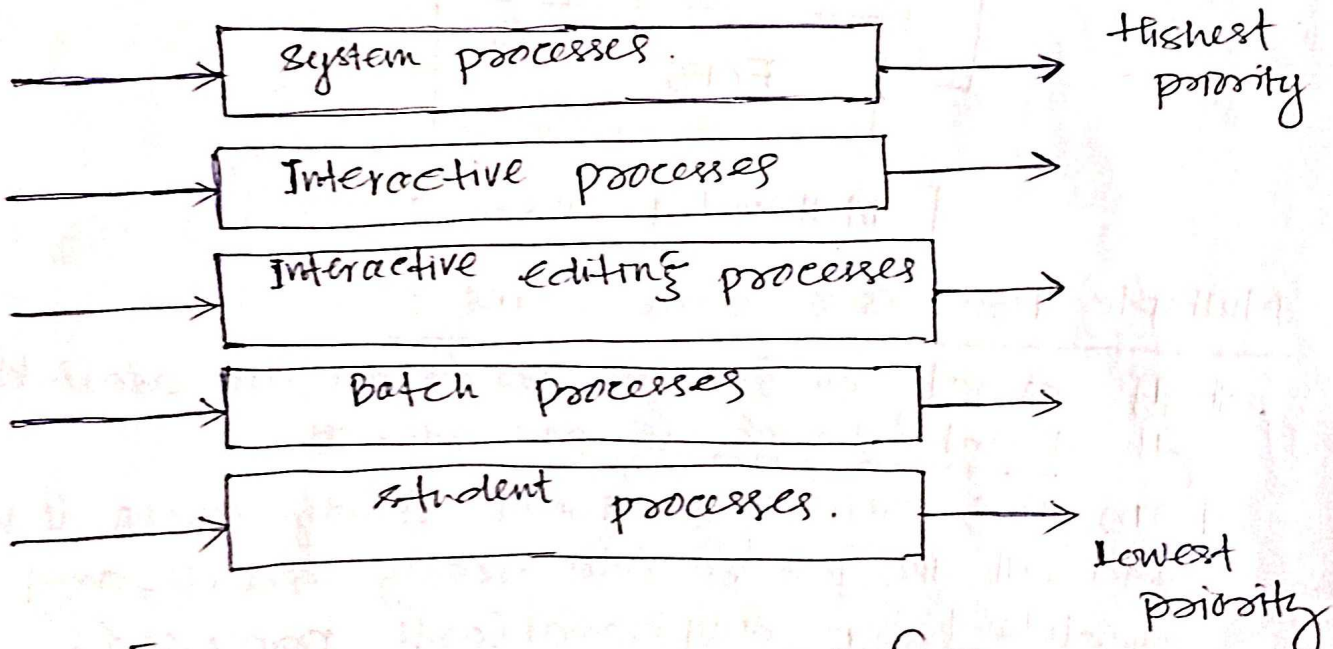
Multilevel Queue scheduling :

A multilevel queue scheduling algorithm partitions the ready queue into several separate queues.

The processes are permanently assigned to one queue based on some property of the process such as memory size, process priority or process type.

When the CPU is available there will be a scheduling between the queues, which is fixed-priority preemptive scheduling.

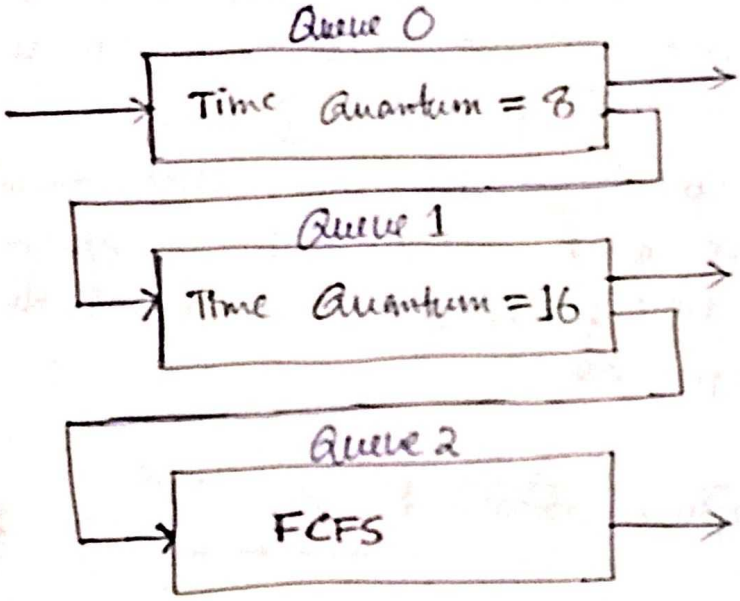
Another possibility is to time-slice among the queues. In this case each queue gets a certain portion of the CPU time, which can then schedule among its various processes.



[Multilevel queue scheduling]

Multilevel Feedback Queue scheduling :

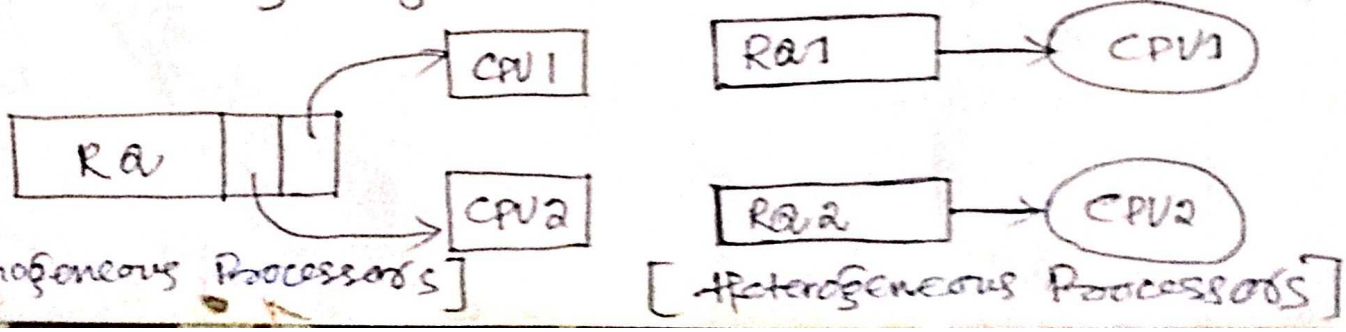
This algorithm allows feedback between the queues.
A job which waits too long in a lower priority queue can be move to a higher priority queue. Similarly if a job uses too much of CPU time than it can be move to a lower priority queue.



[Multilevel Feedback Queues]

Multiple Processor scheduling :

If several homogeneous processors are available the load sharing is possible.
In this case a common ready queue is used and all the job go into ready queue, and are scheduled to any available processor.
If the processor are heterogeneous then each processor will have its own queue and own scheduling algorithm.



[Homogeneous Processors] [Heterogeneous Processors]

Interprocess communication :

daemon ①

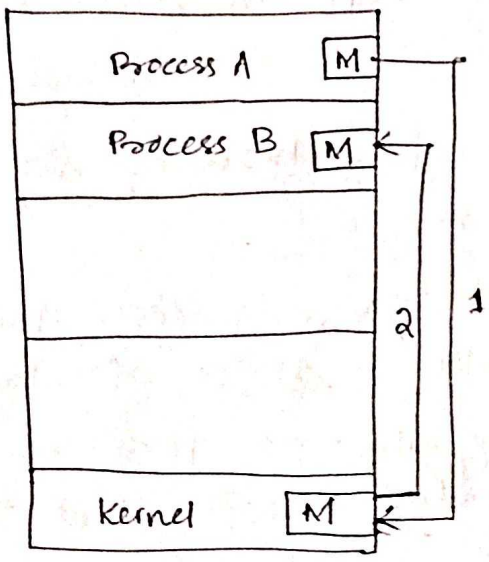
- Process executing concurrently in the OS system may be either independent process or co-operating processes.
- A process is independent if it cannot affect or be affected by the other processes executing in the system.
- A process is cooperating if it can affect or be affected by the other processes executing in the system. ✓
- Cooperating processes require an interprocess communication mechanism that will ensure them to exchange data and information.
- There are two fundamental models of interprocess communication.

(1) shared memory.
(2) message passing.

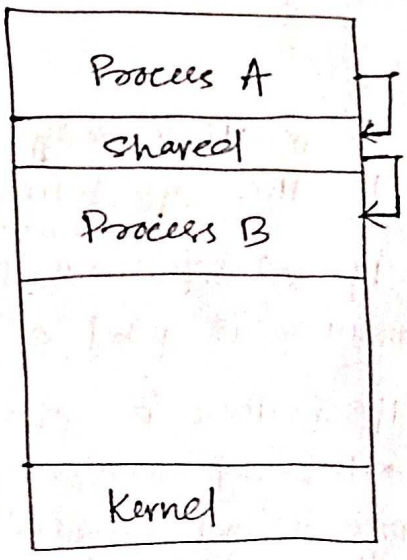
(1) Information sharing
(2) Computational Speedup
(3) Modularity
(4) Convinience

(1) shared memory :

- Interprocess communication using shared memory requires communicating process to establish a region of shared memory.
- A shared memory region resides in the address space of the process creating the shared-memory.
- Other processes that wish to communicate using shared memory segment must attach it to their address space.



(Message Passing)



(Shared memory)

② Message Passing :

- Message passing provides a mechanism to allow processes to communicate and synchronize their action without sharing the same address space and useful in distributed environment.
- A message passing system must facilitate at least two operations : send (message) and receive (message).
- If process P and Q want to communicate, they must send to and receive message from each other over a communication link.
- There are several methods for logically implementing a link and send()/receive() operations :
  - (1) Direct or indirect communication.
  - (2) Synchronous or asynchronous communication.
  - (3) Automatic or explicit buffering.

(1) Direct or Indirect communication :

- Under direct communication, each process want to communicate must explicitly name the recipient or sender.
  - send (P, message) - send a message to process P
  - receive (Q, message) - receive a message from Process Q.
- Under indirect communication the message are sent to and receive from mailboxes or ports.
  - send (A, message) - send a message to mailbox A
  - receive (A, message) - receive a message from mail box A.
- A mailbox may be owned either by a process or by the operating system.
- If a mailbox is owned by a process then the mailbox is part of the address space of the process.
- If mailbox is owned by the OS then it is not attached to any particular process. The OS must provide mechanism to
  - (i) create a new mailbox
  - (ii) send and receive message through mailbox.
  - (iii) Delete a mailbox.

(2) Synchronous or asynchronous communication:

- message passing may be either blocking or non blocking also known as synchronous and asynchronous.

(1) Blocking send: The sending process is blocked until the message is received by the receiving process or by the mailbox.

(2) Non blocking send: The sending process sends the message and resumes operation.

(3) Blocking receive: The receiver blocks until a message is available.

(4) Non blocking receive: The receiver retrieves either a valid message or a null.

(3) Buffering:

whether communication is direct or indirect, message exchanged by communicating process is resside in a temporary queue.

Basically such queues are implimented in three ways.

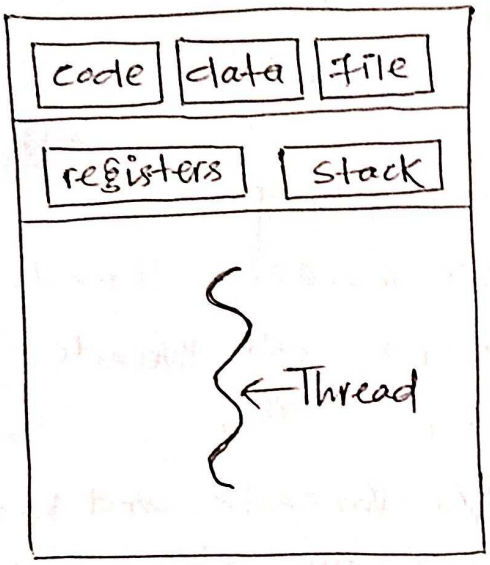
(1) Zero capacity: The queue has maximum length of zero.

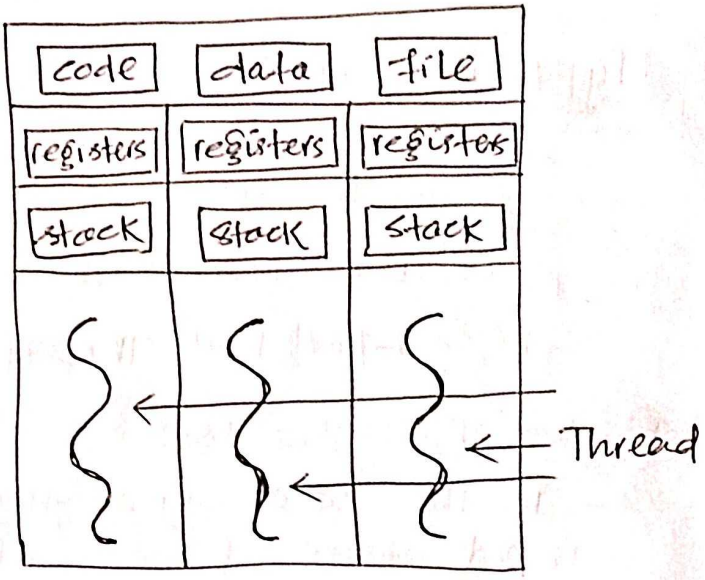
(2) Bounded capacity: The queue has finite length n, thus at most n messages can reside in it.

(3) Unbounded capacity: The queue length is potentially infinite, thus any number of messages can wait in it. The sender never blocks.

Thread

- A thread is a flow of execution through the process code, with its own program counter, system registers and stack.
- A thread is also called a light weight process.
- Each thread belongs to exactly one process and no thread can exist outside a process.
- Each thread represents a separate flow of control.





Process

1. Process is heavy weight or resource intensive.
2. If a process is blocked then that process can not execute untill it is unblocked.
3. In multiple processes each process operate independently of the others.
4. Process switching leads to high overhead on operating system.

Thread

1. Thread is light weight taking lesser resources than a process.
2. While one thread is blocked other thread can be execute belongs to that process.
3. One thread can read, write or change another threads data.
4. Thread switching leads less overhead on OS.

Advantages

1. Thread minimizes context switching time.
2. Use of threads provides concurrency within a process.
3. Efficient communication.
4. Economy - It is more economical to create and content switch threads.
5. Utilization of multiprocessor architecture to a greater scale and efficiency.

Types of Thread

Threads are implimented in two ways.

1. User level Threads - User manages threads.
2. Kernel level Threads - OS manages threads.

1. User level Threads :

- In this case, application manages threads and kernel is not aware of the existance of threads.
- The thread library contains code for creating and destroying threads.

2. Kernel level Threads :

- In this case, thread management is done by the kernel.
- Kernel threads are supported directly by OS.
- The kernel maintains content information for the process as a whole and individual thread within the process.
- Kernel performs thread creation, scheduling and management in kernel space.

Difference between User level & Kernel level Thread:

User level Thread	Kernel level Thread
1. User level threads are faster to create and manage	1. Kernel level threads are slower to create and manage.
2. Implementation is done by a thread library at the user level.	2. Operating system supports creation of Kernel threads.
3. User level thread is generic and can run on any os.	3. Kernel level thread is specific to the os.
4. Multi-threaded application cannot take advantages of multiprocessing	4. Can take advantage of multiprocessing in multiprocessor environment.

Multithreading Models:

Multithreading models are three types.

1. Many to many relationship.
2. many to one relationship.
3. one to one relationship.

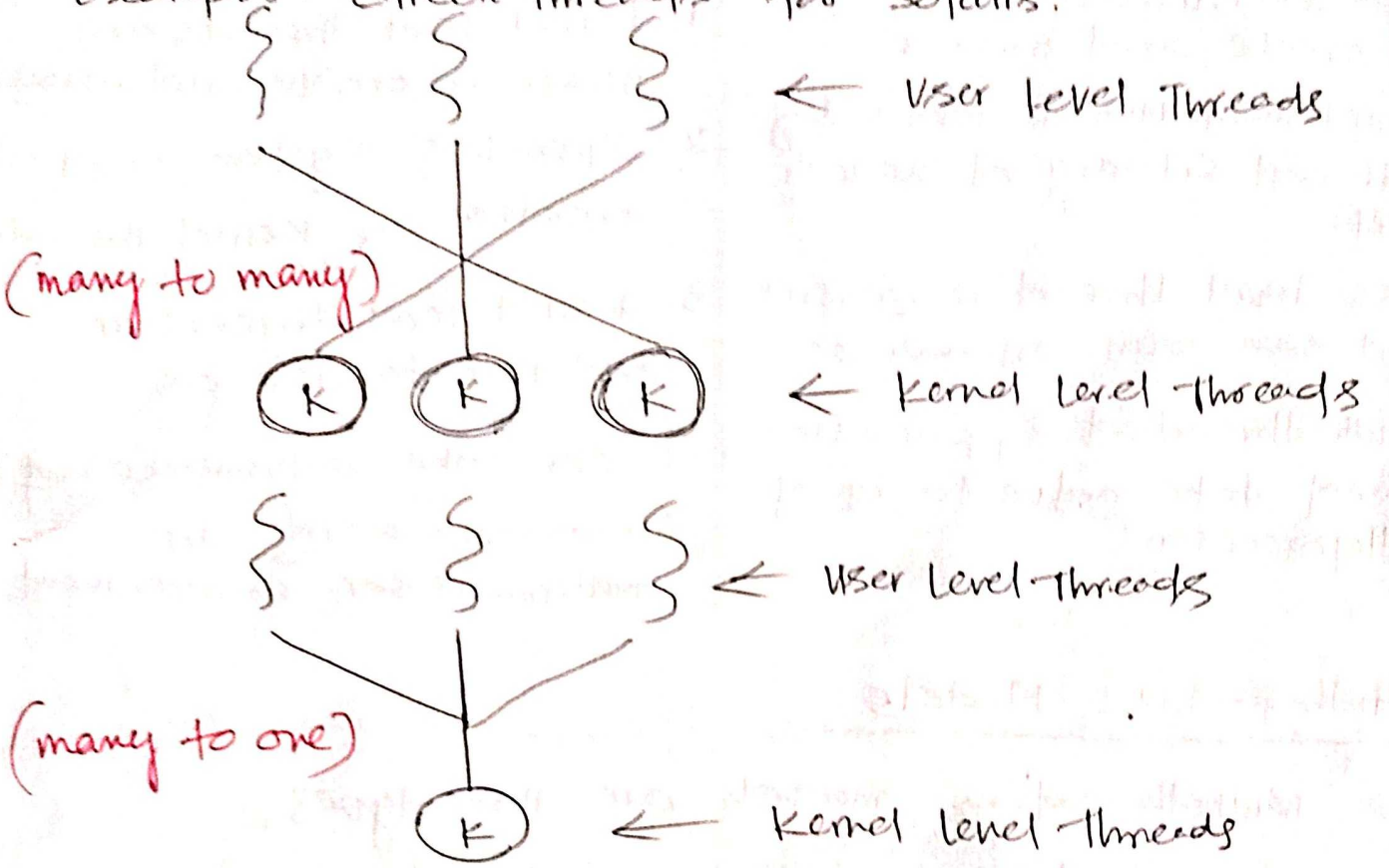
(1) Many to many relationship:

- In this model, many user level threads multiplexes to the Kernel thread of smaller or equal numbers.
- In this model, developers can create as many user threads as necessary and the corresponding Kernel threads can run in parallels on a multiprocessor
- Examples: HP-UX, True64 UNIX.

(2) Many to one relationship:-

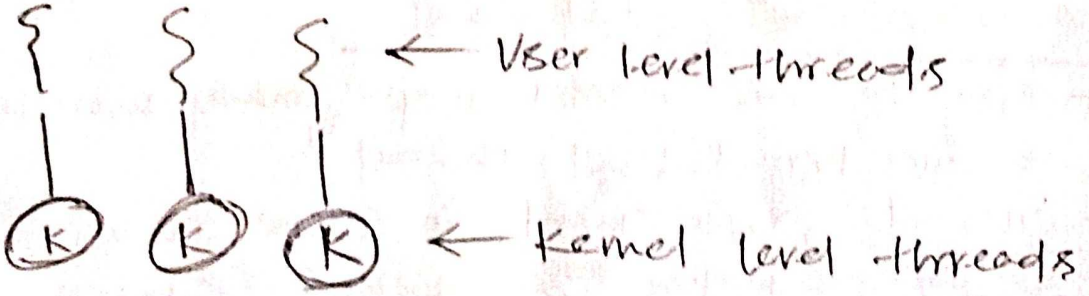
- Many to one model maps many user level threads to one Kernel level thread.
- Thread management is done in user space.
- When a makes a blocking system call,

The entire process will be blocked.
- only one thread can access the kernel at a time, so multiple threads are unable to run in parallel on multiprocessors.
- Examples: Green threads for Solaris.



③ One to one relationship:

- In this model there is one to one relationship between user level thread to kernel level thread.
- This model allows another thread to run when a thread makes a blocking system call.
- It support multiple thread to execute in parallel on microprocessor.
- Disadvantage of this model is that creating a user thread requires corresponding kernel thread.
- Windows family follows one to one relationship.



Deadlock

① System Models

Computer system are full of resources that can only be used by one process at a time. A resource can be a hardware device or a piece of information. A computer will normally have many different resources that can be acquired. Some examples are CPU cycles, files and I/O devices etc.

Resources come in two types : (I) preemptable and (II) nonpreemptable.

A preemptable resource is one that can be taken away from the process without any side effect. Main Memory is an example of a preemptable resource.

A nonpreemptable resource, in contrast is one that cannot be taken away from its owner process without causing the computation to fail. CD recorder are not preemptable resource.

A process must request a resource before using it and must release the resource after using it. Under the normal mode of operation, a process may utilize a resource in the following sequence:

Request : If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.

Use : The process can operate on the resource.

Release : The process release the resource.

In a multiprogramming system several processes compete for finite number of resources. If a process request for resources and resources are not available at that time then the processes enters into a waiting state which may never change the state again, because the requested resources are held by other waiting processes. Such a situation is called deadlock.

Definition:

"A set of processes is in a deadlock state when every process in the set is waiting for an event that can only caused by another processes in the set."

(2) Deadlock Characterization:

Necessary conditions for deadlocks:

1. **Mutual exclusion:** Only one process at a time can use the resource.
2. **Hold and wait:** There must exists a process i.e holding at least one resource and waiting to acquire additional resources that are currently being held by the processes.
3. **No preemption:** Resources can not be preempted. They can only be released voluntarily.
4. **Circular wait:** There must exists a set of $\langle P_0, P_1 \cdots P_{n-1} \rangle$ waiting processes, so that:

$P_0 \rightarrow P_1 \rightarrow P_2 \cdots \rightarrow P_{n-1} \rightarrow P_0$ (waits for)

If each resource have exactly one instance than cycle implies that a deadlock has occured. If each resource has several instance than a cycle doesnot implies a deadlock situation.

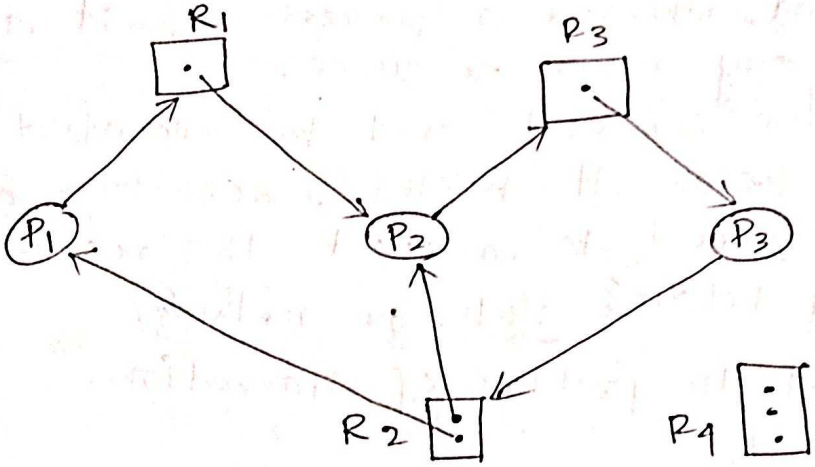
Resource-Allocation Graph :

Resource allocation graph (RAG) consists of a set of vertices V and a set of edges E.

The set of vertices V is partitioned into two diff. types of nodes $P = \{P_1, P_2 \cdots P_n\}$ the set of active processes and $R = \{R_1, R_2 \cdots R_n\}$ the set of all resource types

A directed edge $P_i \rightarrow R_j$ is called a request edge and a directed edge $R_j \rightarrow P_i$ is called assignment edge.

Each process is represented as circle and resource type as a rectangle. If resource type has more than one instance than each instance is represented as a dot within the rectangle.



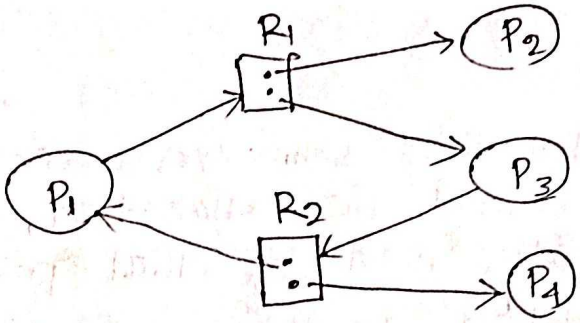
If a resource allocation graph does not have a cycle, then the system is not in a deadlock state.

If there is a cycle then the system may or may not be in deadlock state.

cycles : $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$

[A resource allocation graph with a deadlock]



[Resource allocation graph with a cycle but no deadlock]

(3) Methods for handling deadlock:

(1) Never enter a deadlock:
- Deadlock prevention
- Deadlock avoidance

(2) Enter a deadlock and then recover:
- Deadlock detection
- Deadlock recovery.

(3) Ignore the problem and pretend that deadlock never occurs in the system.

Deadlock Prevention:

By ensuring at least one of the four conditions cannot hold, the occurrence of deadlock can be prevented.

(II) ~~Mutual exclusion~~ Hold and wait: # This condition will never hold if we can ensure that whenever a process request a resource it doesnot hold any other resource.

A process can request and be allocated all its resources before it begins execution or a process is required to request the resource only when if it holding null or nothing.

This may lead to problem of starvation.

(I) Mutual exclusion:

By using only sharable resources:
i.e using only read only files etc.

Unfortunately most of the resources are unsharable.

(III) No-preemption:

If a process is holding some resource and request another resource that cannot be allocated, then all the resources currently held by that process are preempted and added to the pool of resources.

Alternatively we can preempt from waiting processes and allocate them to the requesting process.

(iv) Circular wait :

To break this condition we can impose a total ordering on the use of the resource types. Each process can request the resource in an increasing order of their functional value.

A process can initially request any number of instances of a resource type - let R_i. After that the process can request instances of resource type R_j if and only if $F(R_i) < F(R_j)$.

Alternatively a process is required to ~~release~~ request for R_j only when it has released resources R_i.

Proof by contradiction :

let $\langle P_0, P_1 \cdots P_{n-1} \rangle$ in circular wait.

$P_0 \rightarrow R_0 \rightarrow R_1 \rightarrow R_2 \cdots P_i \rightarrow R_i \rightarrow P_{i+1} \rightarrow R_{i+1} \rightarrow \cdots P_{n-1} \rightarrow R_{n-1}$ (circular)

we know $F(R_i) < F(R_{i+1})$

$\Rightarrow F(R_0) < F(R_1) < \cdots < F(R_i) < F(R_{i+1}) < \cdots < F(R_{n-1}) < F(R_0)$

$\Rightarrow F(R_0) < F(R_0)$ This is impossible.

This protocol often applied to resources whose state can be easily saved and restored later such as CPU registers and memory space.

It cannot generally be applied to resources such as mutex locks and semaphores.

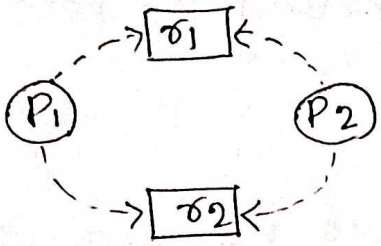
Deadlock Avoidance :

- This method requires information about how the resources are to be requested. With this knowledge for each process, the OS can decide, for each request whether or not the process should wait.
- Each request requires that the system consider the resources currently available, resources currently allocated to each process and the future request and release of each process, to decide if the current request can be satisfied or the process must wait to avoid a future possible deadlock situation.
- The method of deadlock avoidance required that each process declare the maximum no of resources of each type that it may need and ensures that the system will never enter a deadlock state.
- It dynamically examine the resource allocation state to ensure that they can be never in a circular wait condition.
- The allocation state is safe if the system can allocate resources to each process upto its maximum need in some order and avoid future possible deadlock.
- Whenever a process request a resource that is currently available, the OS must decide if the resource can immediately allocated or the process must wait. The request is granted only if it leaves the system is in a safe state.

(i) For single instance of resource type:

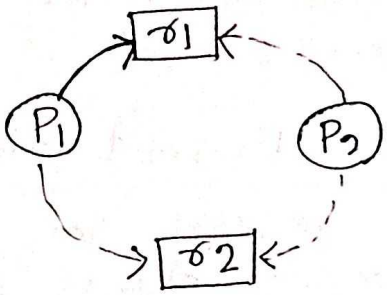
RAG (Resource Allocation Graph)

$G = \{V, E\}$, $V = \{P, R\}$, $E = \{C, R, A\}$



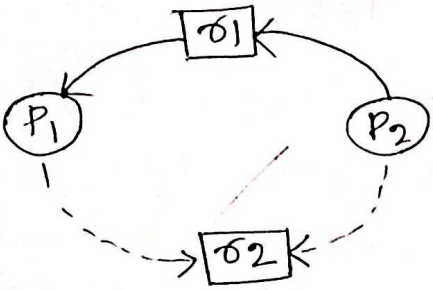
Request:

① $(P_1 \rightarrow r_1)$

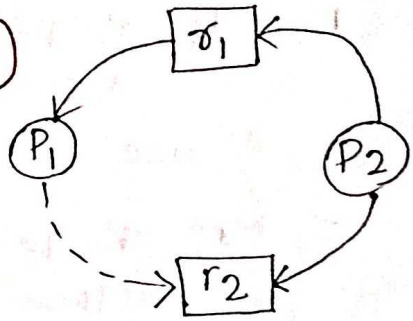


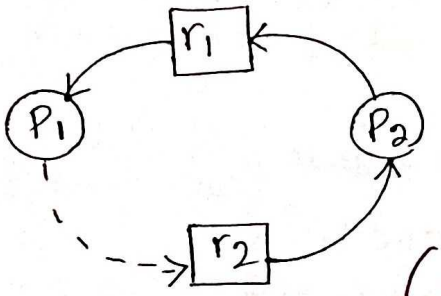
No cycle so allocate the resource.

② $(P_2 \rightarrow r_1)$



③ $(P_2 \rightarrow r_2)$

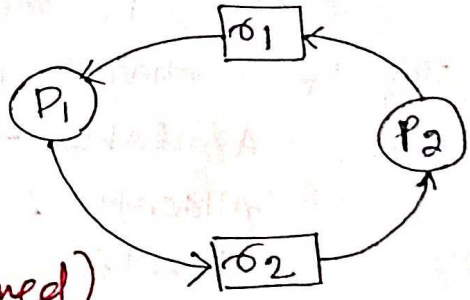




④ $P_1 \rightarrow r_2$

cycle

(If P1 request for r2 then cycle will be formed)



(deadlock)

A claim edge $P_i \rightarrow R_j$ (dotted edges) indicates that process P_i may request resource R_j at some time in future.

When process P_i requests resource R_j then claim edge $P_i \rightarrow R_j$ is converted to a request edge. Similarly when a resource R_j is released by P_i then assignment edge $R_j \rightarrow P_i$ is converted to a claim edge $P_i \rightarrow R_j$.

The request can be granted only if converting the request is to an assignment edge does not result in the formation of cycle in the graph. If no cycle exists then the system is in a safe state otherwise the process will have to wait.

(ii) For several instance of resource types:

The resource-allocation graph algorithm is not applicable to resource allocation system with multiple instances of each resource type. The deadlock avoidance algorithm for multiple instance is called Banker's algorithm.

Resource Request Algorithm:

when a request for resource is made by process P_i following actions are taken.

1. If $Request_i \leq Need_i$
   then goto step 2;
   else "error"
2. If $Request_i \leq Available$
   then goto step 3;
   else "P_i must wait"
3. P_i modifies the states as follows
   $Available = Available - Request_i$
   $Allocation_i = Allocation_i + Request_i$
   $Need_i = Need_i - Request_i$

If the resulting allocation state is safe then transaction is confirmed, else old allocation state is restored.

Safety Algorithm:

1. Initiate temp = Available.
   Finish[i] = false, for all i
2. Find an i such that (a) Finish[i] == false
   (b) $Need_i \leq temp$
   If no such i exist then goto step 4
3. temp = temp + Allocation$_i$
   Finish[i] = true goto step 2
4. If Finish[i] == true for all i
   then the system is in a safe state.

Deadlock Detection :

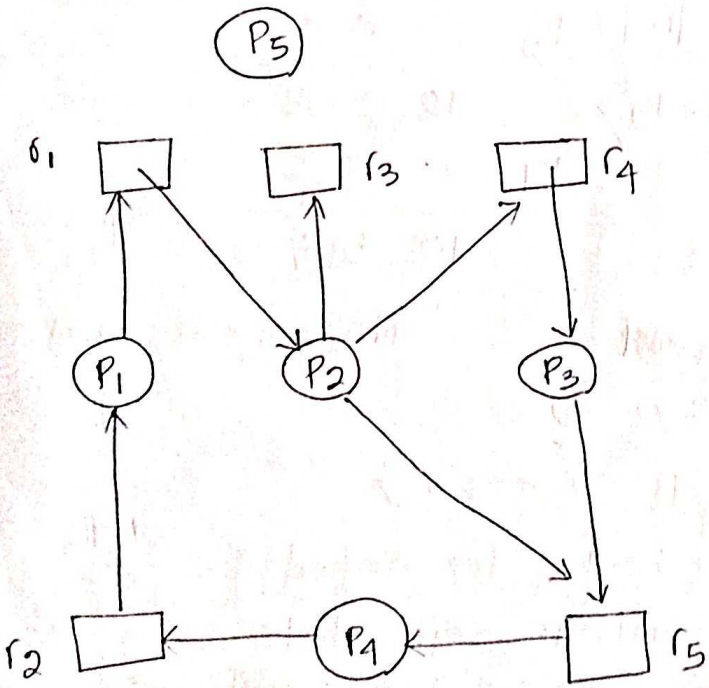
The state of the system is examined periodically to determine whether a deadlock is occured. If so system attempts to recover from the deadlock. For this purpose it has to maintain information about the current resource allocation as well as any outstanding allocation request and provide an algorithm to determine if the system has entered a deadlock state.

(i) For single instance of resource type :

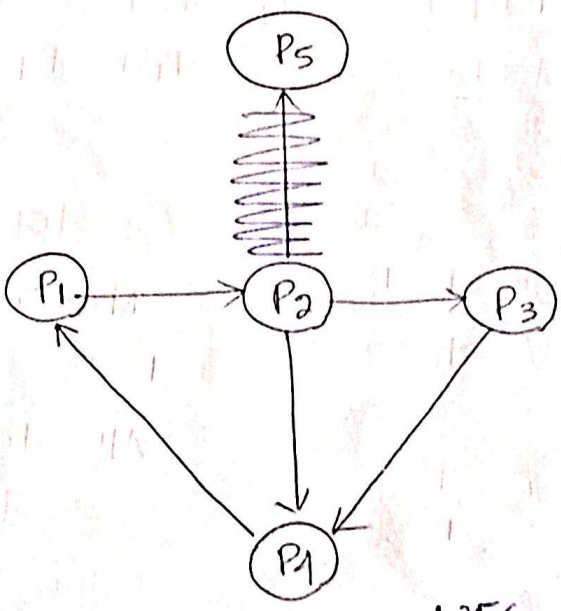
Wait for Graph : Wait for Graph is obtained from the resource allocation Graph by removing the nodes of resource type and corresponding edges.

In this Graph $P_i \rightarrow P_j$ implies that P_i is waiting for P_j to release a resource that it needs. This edge will exists if and only if the original resource allocation Graph contains two edges $P_i \rightarrow r_q$, $r_q \rightarrow P_j$.

A deadlock exists in the system if and only if the wait for Graph contains cycle.



RAG



WFG

Deadlock cycle :

$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

Example :

	Allocation A B C	Max A B C	Need A B C	Available A B C
P_0	0 1 0	7 5 3	7 4 3	3 3 2
P_1	2 0 0	3 2 2	1 2 2	
	~~3 0 2~~		~~0 2 0~~	
P_2	3 0 2	9 0 2	6 0 0	
P_3	2 1 1	2 2 2	0 1 1	
P_4	0 0 2	4 3 3	4 3 1	

Finish

0	1	2	3	4
F	F	F	F	F

$\langle P_1\ P_3\ P_0\ P_2\ P_4 \rangle$

A 10
B 5
C 7
} Resources.

① Step 1 : check for safety.

temp

A	B	C
3	3	2
P_1 2	0	0
5	3	2
P_3 2	1	1
7	4	3
P_0 ~~0~~	~~1~~	~~0~~
~~7~~	5	3
P_2 3	0	2
10	5	5
P_4 0	0	2
10	5	7

Now in safe state.

② Step-2 : Allocate (New request) P_1 Request and

P_1 : $Request_1 = \langle 1,0,2 \rangle$ check :

Available 2 3 0 (3 3 2 − 1 0 2)

Finish

0	1	2	3	4
F	F	F	F	F

$\langle P_1\ P_3\ P_4\ P_0\ P_2 \rangle$

temp

A	B	C
2	3	0
P_1 − 3	0	2
5	3	2
P_3 2	1	1
7	4	3
P_4 0	0	2
7	4	5
P_0 0	1	0
7	5	5
P_2 3	0	2
10	5	7

③ Step-3 :

New request :

P_0 : 0 2 0 ?

Allocate P_0 $\langle 0\ 2\ 0 \rangle$ and check for safety.

This is not in safe state.

So $P_0 \langle 0\ 2\ 0 \rangle$ will not be ~~granted~~. So $P_0 \langle 0\ 2\ 0 \rangle$ will not be allowed.

(This is also in safe state). So $P_1 \langle 1\ 0\ 2 \rangle$ is allowed.

(ii) For several instances of resource types :

Detection Algorithm :

1. Initialize temp = Available

   If Allocation$_i$ $\neq 0$ Finish[i] = False

   else Finish[i] = True , for all i

2. Find an i such that

   Finish[i] = false &&

   Request$_i$ $\leq$ temp

   If no such i exists goto step 4.

3. temp = temp + Allocation$_i$

   Finish[i] = true

   goto step 2.

4. If Finish[i] = false for some i

   then "Deadlock"

Example :

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0	1		
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

P_0	P_1	P_2	P_3	P_4
F	F	F	F	F

T

temp

A	B	C
0	0	0
P_0 - 0	1	0
0	1	0
P_2 - 3	0	3
3	1	3
P_1 - 2	0	0
5	1	3
P_3 - 2	1	1
7	2	4
P_4 - 0	0	2
7	2	6

New Request

$P_2 = \langle 0, 0, 1 \rangle$

Recovery from Deadlock :

There are two options for breaking a deadlock. One solution is to simply abort one or more process-es to break the circular wait. The second option is to preempt some resources from one or more of the deadlock processes.

(1) **Process Termination :** By killing a process the system release the resources allocated to that process.

(i) **Abort all deadlocked process :**

This method will break the deadlock but the processes may have computed for a long time and the results of these partial computation must be discarded and probably re computed later.

(ii) Abort one process at a time until the deadlock cycle is eliminated :

This method incures considerable overhead because after each process is aborted a deadlock detection algorithm will be invoked to determine deadlock.

(2) **Resource preemption :** The resource can be pre-empted successively from processes and can be allocated to other processes until the deadlock cycle is broken.

For resource pre-emption three issues need to be addressed

(i) **selecting a victime :**

The victime should be so choosen that cost should be minimum.

(ii) **Rollback :**

It will be more effective to rollback the process only as per as necessary to break the deadlock.

For this purpose the system can use various check points to record the state of the process to allow the roll back.

(iii) **Starvation :**

The system must ensure that a process can be select resources only a finite no of time so that there will be no starvation.